

一、Python编程语言介绍

众所周知，Python已成为全球3大主流编程语言之一，与C、Java并驾齐驱。与Java, C这两位编程界大佬长期霸屏编程语言排行榜前列不同的是，Python是近几年才异军突起、爬到了前三的位置。活脱脱一个屌丝逆袭故事。

Very Long Term History							
To see the bigger picture, please find below the positions of the top 10 programming languages of many years back. Please note that these are average positions for a period of 12 months.							
Programming Language	2017	2012	2007	2002	1997	1992	1987
Java	1	1	1	1	14	-	-
C	2	2	2	2	1	1	1
C++	3	3	3	3	2	2	4
C#	4	4	7	14	-	-	-
Python	5	7	6	9	27	-	-
PHP	6	5	4	5	-	-	-
JavaScript	7	9	8	7	20	-	-
Visual Basic .NET	8	21	-	-	-	-	-
Perl	9	8	5	4	4	11	-
Assembly language	10	-	-	-	-	-	-
COBOL	25	31	17	6	3	13	3
Lisp	31	12	14	10	9	9	2
Prolog	33	37	26	13	18	14	3
Pascal	102	13	19	29	8	3	5

发展越来越好的

没落的语言

世界上的编程语言有600多种，但真正大家主流在使用的最多二三十种，不同的语言有自己的特点和擅长领域，随着计算机的不断发展，新语言在不断诞生，也同时有很多老旧的语言慢慢无人用了。有个权威的语言排名网站，可以看到主流的编程语言是哪些

*2019年2月数据(<https://www.tiobe.com/tiobe-index/>)

Feb 2019	Feb 2018	Change	Programming Language	Ratings	Change
1	1		Java	15.876%	+0.89%
2	2		C	12.424%	+0.57%
3	4	▲	Python	7.574%	+2.41%
4	3	▼	C++	7.444%	+1.72%
5	6	▲	Visual Basic .NET	7.095%	+3.02%
6	8	▲	JavaScript	2.848%	-0.32%
7	5	▼	C#	2.846%	-1.61%
8	7	▼	PHP	2.271%	-1.15%
9	11	▲	SQL	1.900%	-0.46%
10	20	▲▲	Objective-C	1.447%	+0.32%
11	15	▲▲	Assembly language	1.377%	-0.46%
12	19	▲▲	MATLAB	1.196%	-0.03%
13	17	▲▲	Perl	1.102%	-0.66%
14	9	▼▼	Delphi/Object Pascal	1.066%	-1.52%
15	13	▼	R	1.043%	-1.04%
16	10	▼▼	Ruby	1.037%	-1.50%
17	12	▼▼	Visual Basic	0.991%	-1.19%
18	18		Go	0.960%	-0.46%
19	49	▲▲	Groovy	0.936%	+0.75%
20	16	▼▼	Swift	0.918%	-0.88%

1.1 Python为何能逆袭？

答案：还是因为太优秀了哈哈。

评判一个编程语言的优劣有多种维度，一般包括“开发效率”、“学习曲线”、“生态圈”、“运行速度”等，Python在“开发效率”、“学习曲线”、“生态圈”这3个维度上可以说是稳拿第一了。

开发效率——由于语法简洁明了，又有丰富的、现成的各种模块库，开发什么功能都不用重新造轮子，直接在前辈的写好的代码基础上扩展即可，大大提高开发效率。

学习曲线——Python算是最适合小白上手的语言了。很多人觉得计算机难学，是因为一开始就没选对语言，你一上来就搞C\C++\Java啥的，那指针呀、链表呀、面向对象呀就够你喝一壶的。而学Python符合人性的学习曲线，由简入繁，先易后难。

生态圈——Python自1989年诞生至今30余年，在IT技术的各个领域方向都发展出强大的生态圈，在人工智能、数据分析、爬虫开发、游戏开发、自动化运维\测试、web开发、后端开发、科学运算、GUI编程等方面均全面开花，被称为编程界的瑞士军刀、万能语言。

运行速度——Python的运行速度相比C语言确实慢很多，跟JAVA相比也要慢一些，因此这也是很多所谓的大牛不屑于使用Python的主要原因，但其实这里所指的运行速度慢在大多数情况下用户是无法直接感知到的，必须借助测试工具才能体现出来，比如你用C运一个程序花了0.01s,用Python是0.1s,这样C语言直接比Python快了10倍,算是非常夸张了，但是你是无法直接通过肉眼感知的，因为一个正常人所能感知的时间最小单位是0.15-0.4s左右，哈哈。其实在大多数情况下Python已经完全可以满足你对程序

速度的要求，除非你要写对速度要求极高的搜索引擎、网络游戏等，这种情况下，当然还是建议你用C去实现的。

综上所述，导致Python必火无疑。

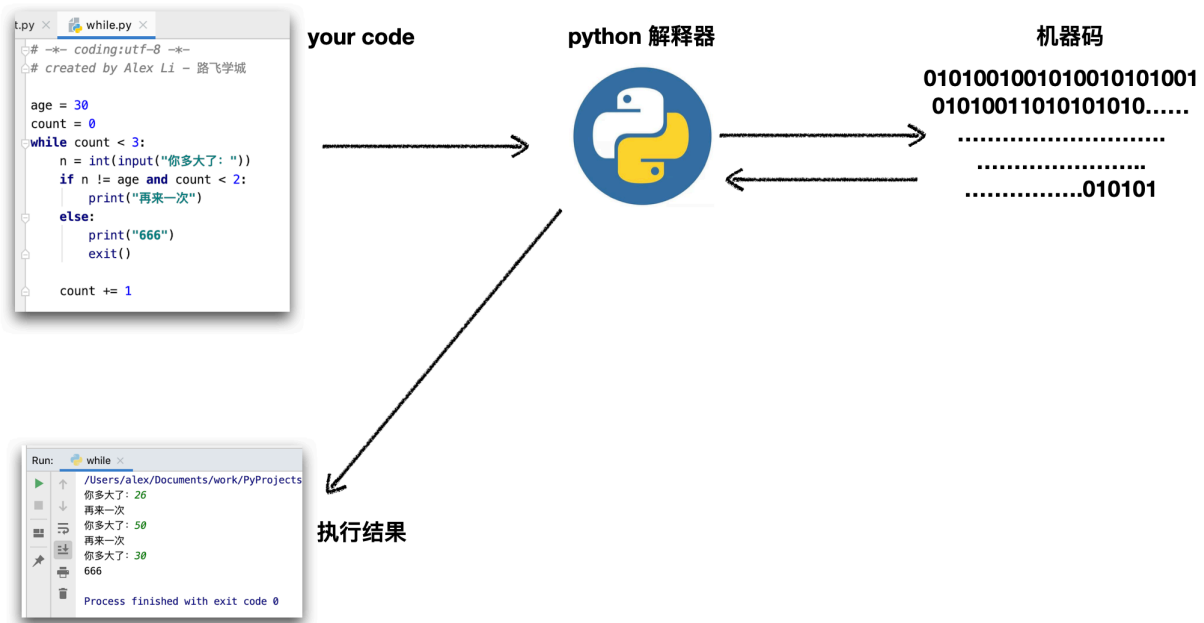
1.2 Python主要应用领域

1. WEB开发——最火的Python web框架Django, 支持异步高并发的Tornado框架，短小精悍的flask,bottle, Django官方的标语把Django定义为the framework for perfectionist with deadlines(大意是一个为完美主义者开发的高效率web框架)
2. 网络编程——支持高并发的Twisted网络框架， py3引入的asyncio使异步编程变的非常简单
3. 爬虫——爬虫领域，Python几乎是霸主地位，Scrapy\Request\BeautifulSoup\urllib等，想爬啥就爬啥
4. 云计算——目前最火最知名的云计算框架就是OpenStack,Python现在的火，很大一部分就是因为云计算
5. 人工智能、数据分析—— Python 是目前公认的人工智能和数据分析领域的必备语言
6. 自动化运维——问问你认识的每个运维人员，运维人员必须会的语言是什么？10个人相信会给你一个相同的答案，它的名字叫Python。知名的运维工具Ansible\Saltstack都是Python写的
7. 金融分析——我个人之前在金融行业，10年的时候，我们公司写的好多分析程序、高频交易软件就是用的Python,到目前,Python是金融分析、量化交易领域里用的最多的语言
8. 科学运算—— 97年开始，NASA就在大量使用Python在进行各种复杂的科学运算，随着NumPy, SciPy, Matplotlib, Enthought librarys等众多程序库的开发，使的Python越来越适合于做科学计算、绘制高质量的2D和3D图像。和科学计算领域最流行的商业软件Matlab相比，Python是一门通用的程序设计语言，比Matlab所采用的脚本语言的应用范围更广泛
9. 游戏开发——在网络游戏开发中Python也有很多应用。比如知名的游戏<文明>、网易公司的《大话西游》就是用Python写的

二、安装Python环境

2.0为何要装Python环境？

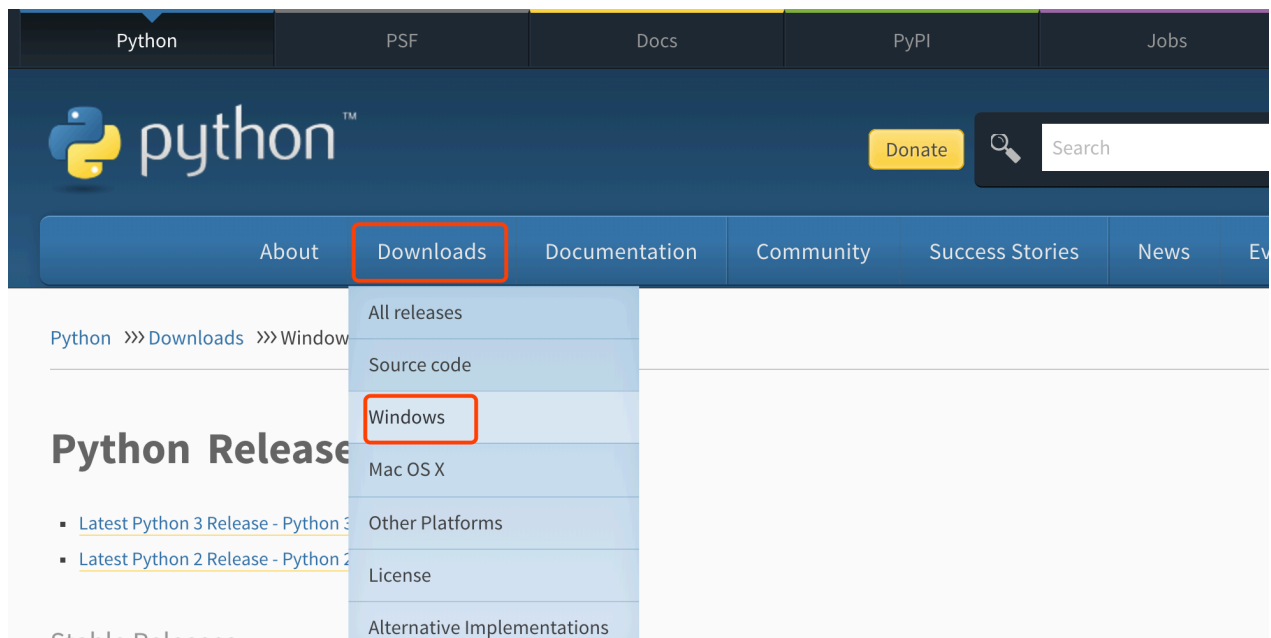
因为你写的代码是人类可读的，但计算机是靠电流驱动的，它只认识010100101，我们即将安装的python环境就是Python解释器，它就像个翻译官，把人类代码翻译成机器能读懂的010101二进制代码，这样才能运行哈。



Python目前已支持所有主流操作系统，在Linux,Unix,Mac系统上自带Python环境，在Windows系统上需要安装一下，超简单。

2.1 Windows安装Python

打开官网 <https://www.python.org/downloads/windows/> 下载中心



Python Releases for Windows

- [Latest Python 3 Release - Python 3.8.1](#)
- [Latest Python 2 Release - Python 2.7.17](#)

Stable Releases

选最新的稳定版本

- [Python 3.8.1 - Dec. 18, 2019](#)

Note that Python 3.8.1 cannot be used on Windows XP or earlier.

- Download [Windows help file](#)
- Download [Windows x86-64 embeddable zip file](#)
- Download [Windows x86-64 executable installer](#)
- Download [Windows x86-64 web-based installer](#)
- Download [Windows x86 embeddable zip file](#)
- Download [Windows x86 executable installer](#)
- Download [Windows x86 web-based installer](#)

选64位的安装文件

- [Python 3.7.6 - Dec. 18, 2019](#)

Note that Python 3.7.6 cannot be used on Windows XP or earlier.

- Download [Windows help file](#)

下载后执行，点击下一步安装就行，注意选择添加Python到当前用户环境变量里



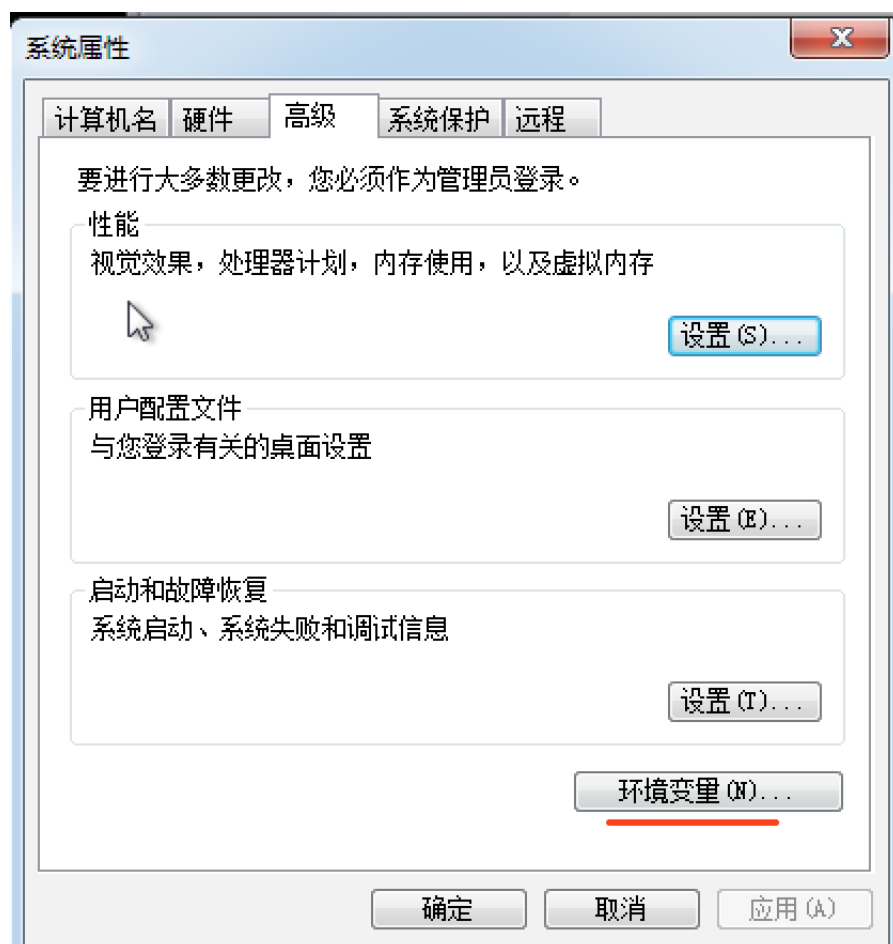
详细安装文档: <https://www.cnblogs.com/wumac/p/8195025.html>

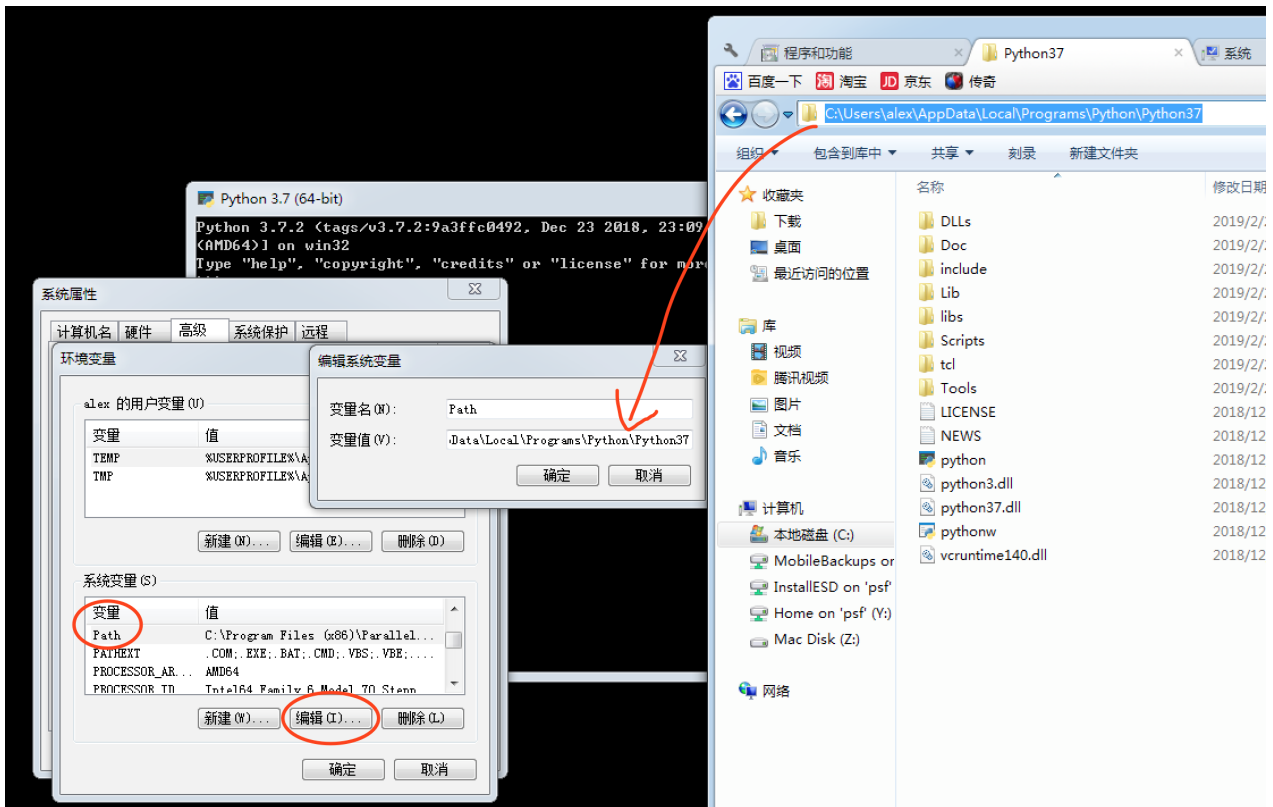
2.2 Windows配置环境变量（选做）

如果在安装Python时没选择环境变量，你是没办法在操作系统的任意目录调用 Python的，必须手动把Python的执行程序加到环境变量里。

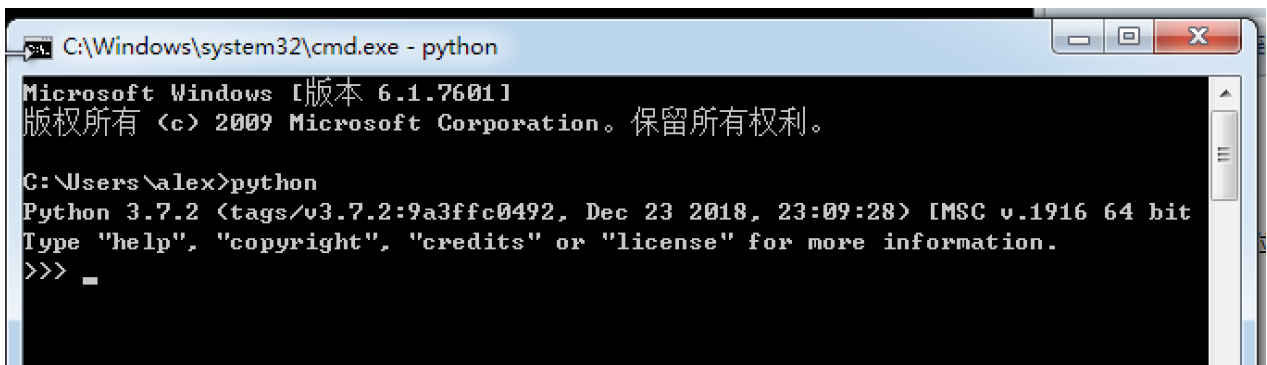
我的Python刚才装到C:\Users\alex\AppData\Local\Programs\Python\Python37 目录下了，以后每次执行python程序时，还要到这个目录下调用python.exe这个可执行文件才行。太麻烦了，为了方便调用，可以配置系统的环境变量，让你的Python可以很容易的被找到。

找到我的计算机，右单击——》属性——》环境变量





windows —> 运行 —> 输入cmd，然后回车，弹出cmd程序，输入python,如果能进入交互环境，代表环境变量成功。



2.3 Mac 安装Python

首先Mac是自带Python的，只要打开终端Terminal输入Python即可，但你若想安装最新版本，如下

Python >>> Downloads >>> Mac OS X

Python Releases for Mac OS X

- [Latest Python 3 Release - Python 3.8.1](#)
- [Latest Python 2 Release - Python 2.7.17](#)

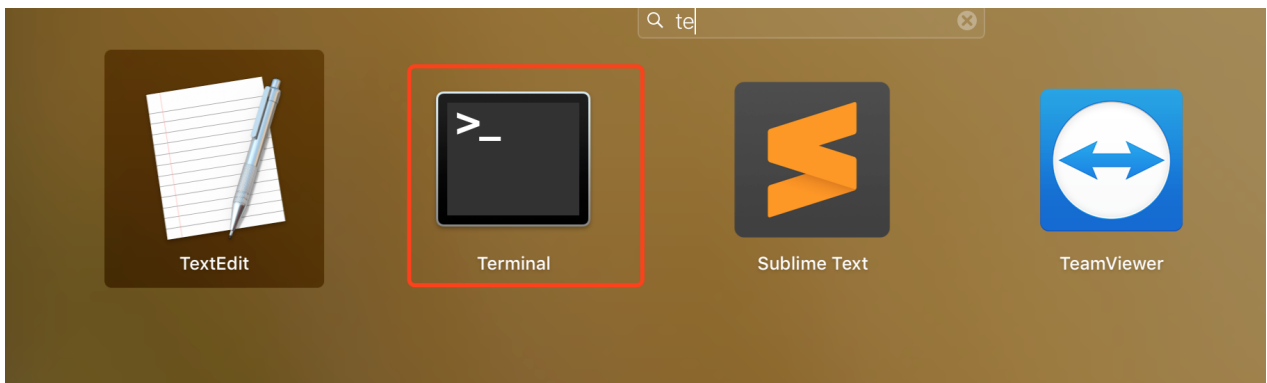
Stable Releases

- [Python 3.8.1 - Dec. 18, 2019](#)
 - Download [macOS 64-bit installer](#)
- [Python 3.7.6 - Dec. 18, 2019](#)
 - Download [macOS 64-bit/32-bit installer](#)
 - Download [macOS 64-bit installer](#)
- [Python 3.6.10 - Dec. 18, 2019](#)
 - No files for this release.
- [Python 3.5.9 - Nov. 2, 2019](#)
 - No files for this release.

Pre-releases

- [Python 3.9.0a3 - Jan. 24, 2020](#)
 - Download [macOS 64-bit installer](#)
- [Python 3.9.0a2 - Dec. 18, 2019](#)
 - Download [macOS 64-bit installer](#)
- [Python 3.7.6rc1 - Dec. 11, 2019](#)
 - Download [macOS 64-bit/32-bit installer](#)
 - Download [macOS 64-bit installer](#)
- [Python 3.6.10rc1 - Dec. 11, 2019](#)
 - No files for this release.

安装完成后，通过Launchpad找到Terminal命令行终端



输入Python3


```
alex — Python — 80x24
~ — Python

Last login: Wed Feb 20 22:30:31 on ttys001
Alexs-MacBook-Pro:~ alex$ python3
Python 3.6.1 (v3.6.1:69c0db5050, Mar 21 2017, 01:21:04)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

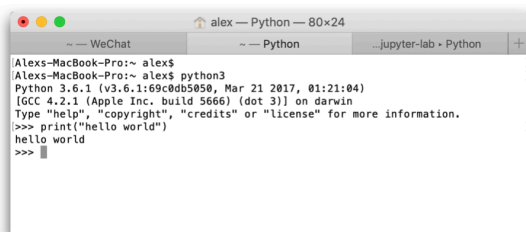
恭喜你，Python安装成功啦。

三、写下你的第一行代码

执行Python代码可以通过2种方式，一种是在Python交互器里直接写，一种是写到文件里保存执行。

Python 交互器

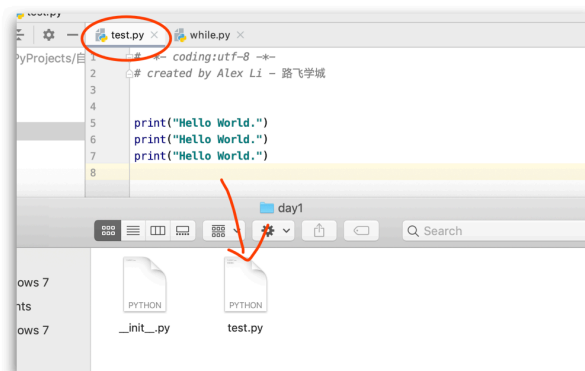
- 退出交互器，代码即丢失，仅供调试使用



```
Alexs-MacBook-Pro:~ alex$ python3
Python 3.6.1 (v3.6.1:69c0db5050, Mar 21 2017, 01:21:04)
[GCC 4.2.1 (Apple Inc. build 5666) (dot 3)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> print("hello world")
hello world
>>>
```

写在文件里

- 可以永久保存，随时执行
- 开发软件都要写在文件里

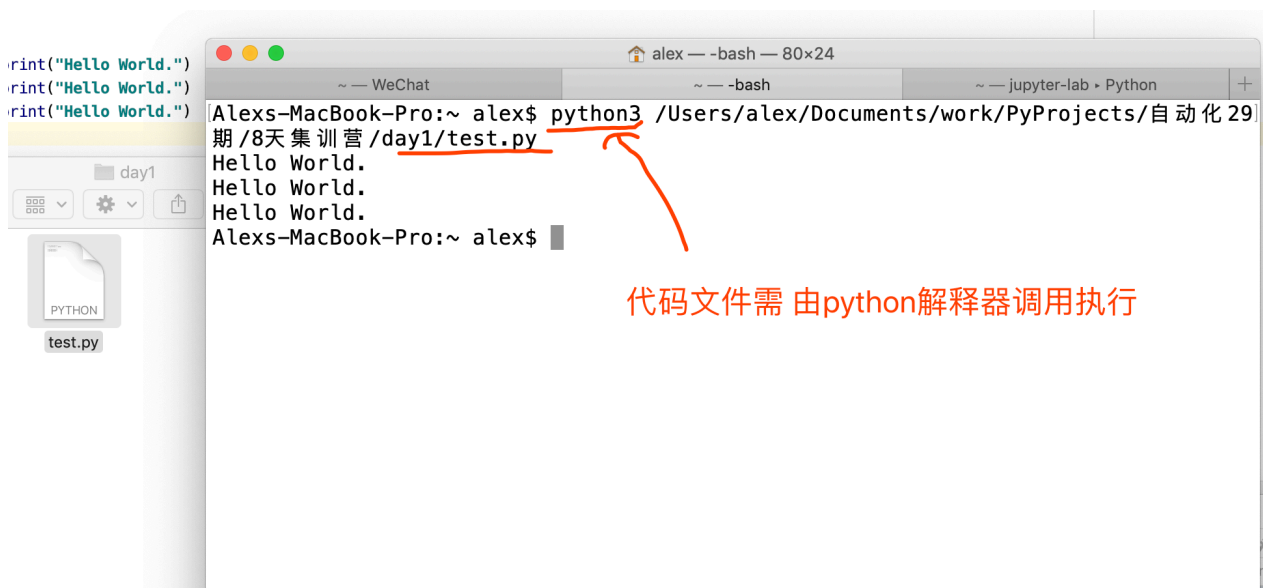


3.1 用交互器执行代码

windows —> 运行 —> 输入cmd，然后回车，弹出cmd程序，输入python，就进入了交互器模式

向屏幕输出一行文字的语法是 `print("括号是你要打印的字，用引号包起来")`。

3.2 代码写在文件里保存执行



四、变量

4.1 什么是变量

变量，是用于在内存中存放程序数据的容器

怎么理解这句话呢？

计算机的最核心功能就是“计算”，CPU是负责计算的，计算需要数据源，数据源要存在内存里，比如我要把Alex的姓名、身高、年龄信息存下来，后面程序会调用，怎么存呢，直接设置一个 `变量名=值`，就可以

定义变量

```
name = "Alex"
age = 22
height = 180
```

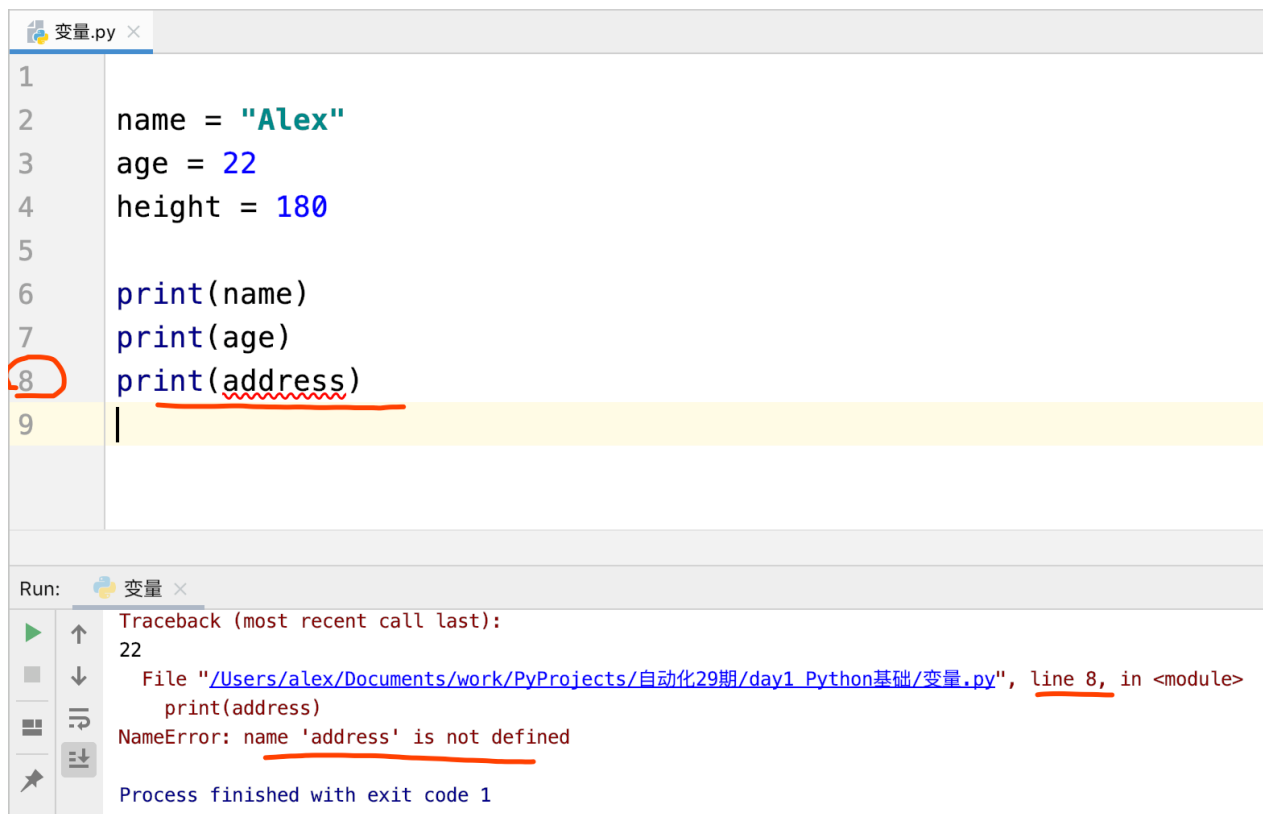
后面程序想调用的时候，直接调 变量名 就可以

```
name = "Alex"
age = 22
height = 180

print(name) # 调用变量
print(age)
```

4.2 变量的使用规则

程序是从上到下依次逐行执行的，所以变量必须先定义，后调用，否则会报错



The screenshot shows a Python IDE with a file named '变量.py'. The code is as follows:

```
1
2 name = "Alex"
3 age = 22
4 height = 180
5
6 print(name)
7 print(age)
8 print(address)
9
```

Line 8 is circled in red, and the variable 'address' is underlined in red. Below the code editor, the 'Run' panel shows a traceback error:

```
Traceback (most recent call last):
  22 File "/Users/alex/Documents/work/PyProjects/自动化29期/day1_Python基础/变量.py", line 8, in <module>
    print(address)
NameError: name 'address' is not defined
Process finished with exit code 1
```

4.3 变量的定义规则

1. 在程序全局都会用到的变量，尽量定义在文件开头
2. 变量名只能是字母、数字或下划线的任意组合，其它形式均不合法
3. 变量名的第一个字符不能是数字
4. 以下关键字不要声明为变量名['and', 'as', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'exec', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'not', 'or', 'pass', 'print', 'raise', 'return', 'try', 'while', 'with', 'yield'], 这些是Python语法中已经占用了的关键字

常用变量名定义规范

驼峰体

```
AgeOfOldboy = 56
NumberOfStudents = 80
c, c#
```

下划线

```
age_of_oldboy = 56
lao_nan_hai_de_nianling # 不要这么写
number_of_students = 80
```

你觉得哪种看着更清晰，哪种就是官方推荐的，我想你肯定会先第2种

定义变量不好的方式

1. 用拼音、中文做变量名
2. 变量名过长
3. 不能让人通过名字就能看出来是什么意思
4. 首字母大写(大写是用来定义类名的, 后面会学)

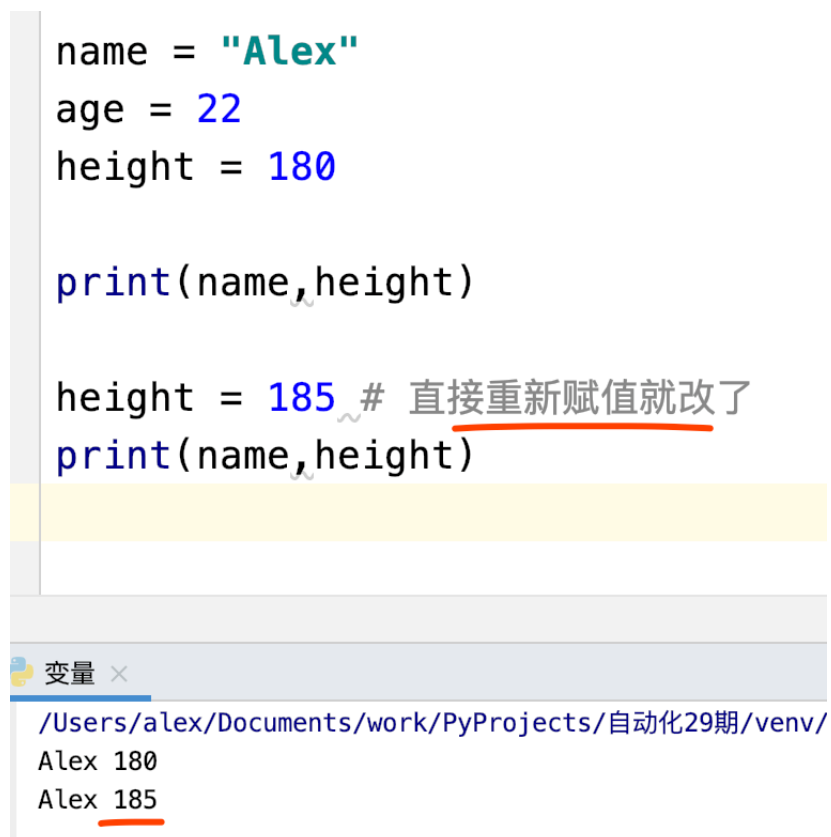
4.4 变量的修改

自行看图不解释

```
name = "Alex"
age = 22
height = 180

print(name,height)

height = 185 # 直接重新赋值就改了
print(name,height)
```



The screenshot shows a Python IDE with a code editor and a terminal window. The code in the editor defines a variable 'name' with the value 'Alex', 'age' with 22, and 'height' with 180. It then prints 'name' and 'height'. After that, 'height' is reassigned to 185, and 'name' and 'height' are printed again. A comment in Chinese says '直接重新赋值就改了' (Directly reassigning changes it). The terminal output shows 'Alex 180' on the first line and 'Alex 185' on the second line, with '185' underlined in red.

del name 删除变量

name = alex
name2 =



五、基本的数据类型

5.1 什么是数据类型？

我们人类可以很容易的分清数字与字符的区别，但是计算机并不能呀，计算机虽然很强大，但从某种角度看又很傻，除非你明确的告诉它，1是数字，“汉”是文字，否则它是分不清1和‘汉’的区别的，因此，在每个编程语言里都会有一个叫数据类型的东东，其实就是对常用的各种数据类型进行了明确的划分，你想让计算机进行数值运算，你就传数字给它，你想让他处理文字，就传字符串类型给他。Python中常用的数据类型包括多种，今天我们暂只讲4种， 数字、字符串、布尔类型、列表。

5.2 数字类型

int (整型) integer

你存的所有的整数，都是这个类型

```
>>> age_alex = 26
>>> alex_salary = 500000000

>>> type(age_alex), type(alex_salary)
(<class 'int'>, <class 'int'>) # 都是int类型
```

可用 `type()` 方法来查看数据类型

float (浮点型)

即小数

```
>>> type(2.32)
<type 'float'>
>>> type(-3.141592653)
<class 'float'>
```

5.3 字符串

在Python中,加了引号的字符都被认为是字符串!

```
>>> name = "Alex Li" #双引号
>>> age = "22"        #只要加引号就是字符串
>>> age2 = 22          #int
>>>
>>> msg = '''My name is Alex, I am 22 years old!''' #我擦, 3个引号也可以
>>>
>>> hometown = 'ShanDong' #单引号也可以
```

那单引号、双引号、多引号有什么区别呢? 让我大声告诉你, 单双引号木有任何区别, 只有下面这种情况 你需要考虑单双的配合

```
msg = "My name is Alex , I'm 22 years old!"
```

多引号

多引号什么作用呢? 作用就是多行字符串必须用多引号

```
msg = '''
今天我想写首小诗,
歌颂我的同桌,
你看他那乌黑的短发,
好像一只炸毛鸡。
'''
print(msg)
```

字符串拼接

数字可以进行加减乘除等运算, 字符串呢? 让我大声告诉你, 也能? what ?是的, 但只能进行“相加”和“相乘”运算。

```

>>> name
'Alex Li'
>>> age
'22'
>>>
>>> name + age #相加其实就是简单拼接
'Alex Li22'
>>>
>>> name * 10 #相乘其实就是复制自己多少次，再拼接在一起
'Alex LiAlex LiAlex LiAlex LiAlex LiAlex LiAlex LiAlex Li'

```

不用尝试相减、相除，臣妾办不到呀。

注意，字符串的拼接只能是双方都是字符串，不能跟数字或其它类型拼接

```

>>> name = "Alex Li"
>>> age = 23
>>>
>>> type(name), type(age)
(<type 'str'>, <type 'int'>)

>>> name + age # 字符串跟数字相加报错
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'int' objects

```

5.4 布尔类型

布尔类型很简单，就两个值，一个True(真)，一个False(假)，主要用作逻辑判断

但其实你们并不明白上面这句话什么意思对么？let me explain, 我现在有2个值，`a=3, b=5`，我说`a>b`你说成立么？我们当然知道不成立，但问题是计算机怎么去描述这成不成立呢？或者说`a < b`是成立，计算机又怎么描述这个公式是成立的呢？

没错，答案就是，用布尔类型

```

>>> a=3
>>> b=5
>>>
>>> a > b #不成立就是False,即假
False
>>>
>>> a < b #成立就是True, 即真
True

```

计算机为什么要描述这种条件呢？因为接下来就可以根据条件结果来干不同的事情啦呀！比如

```
if a > b
    print("a is bigger than b ")
else
    print("a is smaller than b" )
```

上面是伪代码，但是不是意味着，计算机就可以根据判断结果不同，来执行不同的动作啦？

5.5 列表(List)

如果要把全班的人名在内存里存下来，用上面的字符串类型可以做到，但取的时候不方便。

```
names = "Alex,Jack,Rain,Rachel,Mack..."
```

你 `print(names)` 它打印的是所有人的信息，如果想取出Rain，没办法(可以用字符串切割方式，但是很麻烦)。此时，用列表就比较合适。

```
>>> names = ["Alex", "Jack", "Rain", "Rachel", "Mack"]
>>> names[2] #为何names[2]就能取出Rain?
'Rain'
```

因为列表的是通过下标来标记元素位置的，下标也可称为索引。下标从0开始，每添加一个元素，就自动+1

元素名	Alex	Jack	Rain	Rachel	Mack
下标(索引)	0	1	2	3	4

元素添加

元素的添加有2种方式，插入、追加，插入指可以可以把一个新元素插入到列表的任意位置

插入

```
>>> names
['Alex', 'Jack', 'Rain', 'Rachel', 'Mack']
>>> names.insert(3, "小明") #3代表你想插入的下标位置
>>> names
['Alex', 'Jack', 'Rain', '小明', 'Rachel', 'Mack']
>>>
```

追加

添加到列表的尾部


```
>>> names
['Alex', 'Jack', 'Rain', '小明', 'Rachel', 'Mack']
>>> names.append("小强")
>>> names
['Alex', 'Jack', 'Rain', '小明', 'Rachel', 'Mack', '小强']
```

列表的修改

直接根据下标找到元素重新赋值即可

```
>>> names[0] = "金角大王Alex"
>>> names
['金角大王Alex', 'Jack', 'Rain', '小明', 'Rachel', 'Mack', '小强']
```

删除元素

有2种删除方法，一个是下标删除，一个是元素值删除。

```
>>> names
['金角大王Alex', 'Jack', 'Rain', '小明', 'Rachel', 'Mack', '小强']
>>> names.remove("小明") # 通过指定元素名删除，会删除从左开始找到的第一个小明，如果有
多个小明，则只删除找到的第一个。
>>> names
['金角大王Alex', 'Jack', 'Rain', 'Rachel', 'Mack', '小强']

>>> del names[3] # 通过指定下标删除
```

判断某元素是否在列表里

```
>>> names
['金角大王Alex', 'Jack', 'Rain', 'Rachel', 'Mack', '小强']
>>>
>>> "Mack" in names
True
```

- 注，以上只是列表的部分用法，后续我们会深入学习，我们今天先了解这些就够了。

5.6 小练习

1. 一句代码打印10行“Alex金角大王是沙河最靓的仔”，注意是10行。
2. 写一个列表，把你和你身边前后6个同桌的名字存进去，把Alex插入到你后面
3. 把上一题列表里的你自己删除掉，然后再追加到列表尾部

六、运算符

计算机可以进行的运算有很多种，可不只加减乘除这么简单，运算按种类可分为算数运算、比较运算、逻辑运算、赋值运算、成员运算、身份运算、位运算，今天我们暂只学习算数运算、比较运算、逻辑运算、赋值运算、成员运算

6.1 算术运算符

以下假设变量：**a=10, b=20**

运算符	描述	实例
+	加 - 两个对象相加	a + b 输出结果 30
-	减 - 得到负数或是一个数减去另一个数	a - b 输出结果 -10
*	乘 - 两个数相乘或是返回一个被重复若干次的字符串	a * b 输出结果 200
/	除 - x除以y	b / a 输出结果 2
%	取模 - 返回除法的余数	b % a 输出结果 0
**	幂 - 返回x的y次幂	a**b 为10的20次方， 输出结果 100000000000000000000
//	取整除 - 返回商的整数部分	9//2 输出结果 4 , 9.0//2.0 输出结果 4.0

6.2 比较运算符

以下假设变量：**a=10, b=20**

运算符	描述	实例
==	等于 - 比较对象是否相等	(a == b) 返回 False。
!=	不等于 - 比较两个对象是否不相等	(a != b) 返回 true。
<>	不等于 - 比较两个对象是否不相等	(a <> b) 返回 true。这个运算符类似 != 。
>	大于 - 返回x是否大于y	(a > b) 返回 False。
<	小于 - 返回x是否小于y。所有比较运算符返回1表示真，返回0表示假。这分别与特殊的变量 True和False等价。注意，这些变量名的大写。	(a < b) 返回 true。
>=	大于等于 - 返回x是否大于等于y。	(a >= b) 返回 False。
<=	小于等于 - 返回x是否小于等于y。	(a <= b) 返回 true。

注意:<>在python3里已经取消了，仅在py2里有效

6.3 赋值运算符

以下假设变量：**a=10, b=20**

运算符	描述	实例
=	简单的赋值运算符	c = a + b 将 a + b 的运算结果赋值为 c
+=	加法赋值运算符	c += a 等效于 c = c + a
-=	减法赋值运算符	c -= a 等效于 c = c - a
*=	乘法赋值运算符	c *= a 等效于 c = c * a
/=	除法赋值运算符	c /= a 等效于 c = c / a
%=	取模赋值运算符	c %= a 等效于 c = c % a
**=	幂赋值运算符	c **= a 等效于 c = c ** a
//=	取整除赋值运算符	c //= a 等效于 c = c // a

6.4 逻辑运算

以下假设变量：a=10, b=20

运算符	描述	实例
and	判断多个条件均为真时，返回真	a>10 and b > 10 结果为False
or	判断多个条件任意条件为真时，返回真	a> 10 or b > 10 结果为True
not	取反	not a > b 结果为True

注意了还可以多个or 或and 拼在一起

(1, 3, 10)

```
[>>>
```

```
[>>> a > 0 or b < 3
```

```
True
```

```
>>> (a > 0) or (b < 3 and c > 10)
```

T
N
N

or的优先级更高

```
[>>> a,b,c
```

```
(10, 20, 5)
```

```
[>>> a > b and a < c or b > c and a > b and a > c
```

```
False
```

```
[>>> a > b and a < c or b > c and a > b or a > c
```

```
True
```

```
[>>> (a > b and a < c or b > c and a > b) and a > c
```

```
False
```

```
>>>
```

注意，我画的红线，只要遇到or 就相当于把公式分成2半了

括号里的子公式会先计算出来结果，再跟其它的值对比。因为括号的优先级更高

6.5 成员运算

只有in, not in 2种，用来测试数据中是否包含了一系列的成员

运算符	描述	实例
in	如果在指定的序列中找到值返回True，否则返回False。	x 在 y序列中，如果x在y序列中返回True。
not in	如果在指定的序列中没有找到值返回True，否则返回False。	x 不在 y序列中，如果x不在y序列中返回True。

```
>>> names
[ '金角大王Alex', 'Jack', 'Rain', 'Rachel', 'Mack', '小强' ]
>>> "Jack" in names
True
>>> "Jack" not in names
False
>>> s = "我是沙河最帅的仔"
>>> "沙河" in s
True
```

- 注意，可以用来测试字符串、列表、元组、字典、集合，但是不能测试数字类型

七、全宇宙最好用的代码编辑器

到现在为止，我们也是写过代码的人啦，但你有没有发现，每次写代码要新建文件、写完保存时还要选择存放地点，执行时还要切换到命令行调用python解释器，好麻烦呀，能否一气呵成，让我简单的写代码？此时开发工具IDE集成开发环境（Integrated Development Environment）上场啦，一个好的IDE能帮你大大提升开发效率。

很多语言都有比较流行的开发工具，比如JAVA 的Eclipse, C#,C++的VisualStudio, Python的是啥呢？Pycharm，最好的Python 开发IDE。

7.1 安装Pycharm

下载地址:<https://www.jetbrains.com/pycharm/download> 选择Professional 专业版

可以免费试用30天



Version: 2020.1
Build: 201.6668.115
8 April 2020

[System requirements](#)[Installation Instructions](#)[Other versions](#)

Download PyCharm

[Windows](#) [Mac](#) [Linux](#)

Professional

For both Scientific and Web Python development. With HTML, JS, and SQL support.

[Download](#)[Free trial](#)

Community

For pure Python development

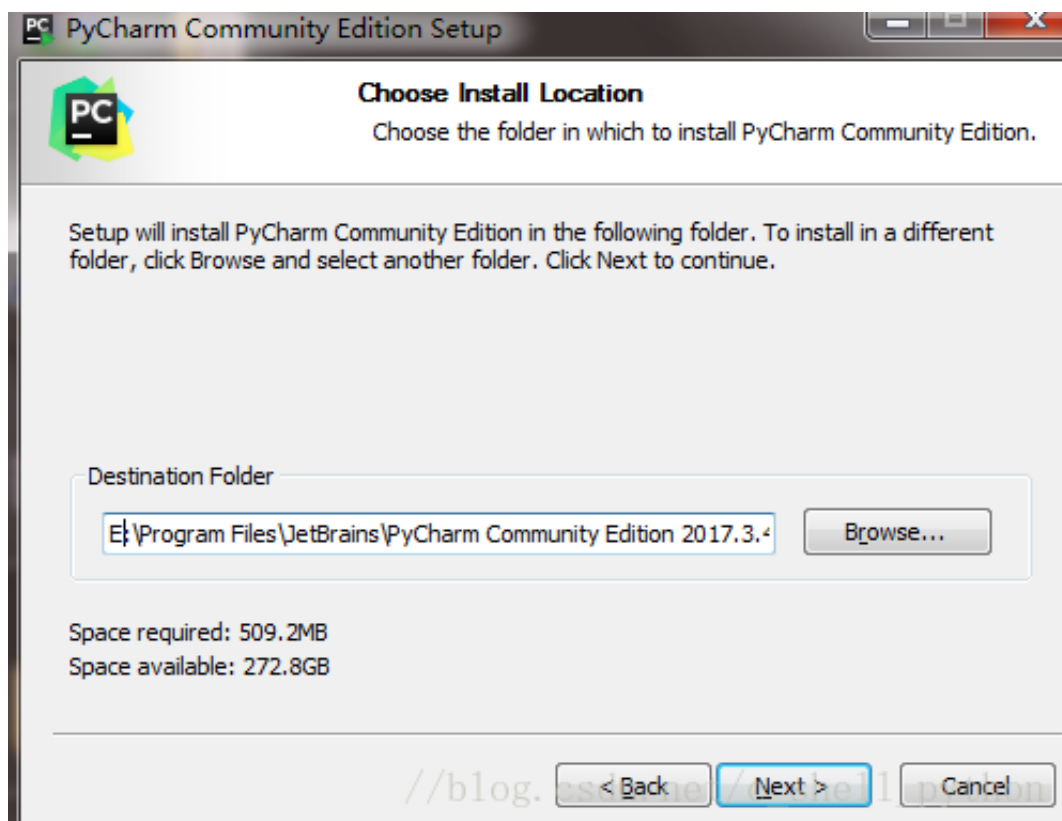
[Download](#)

Free, open-source

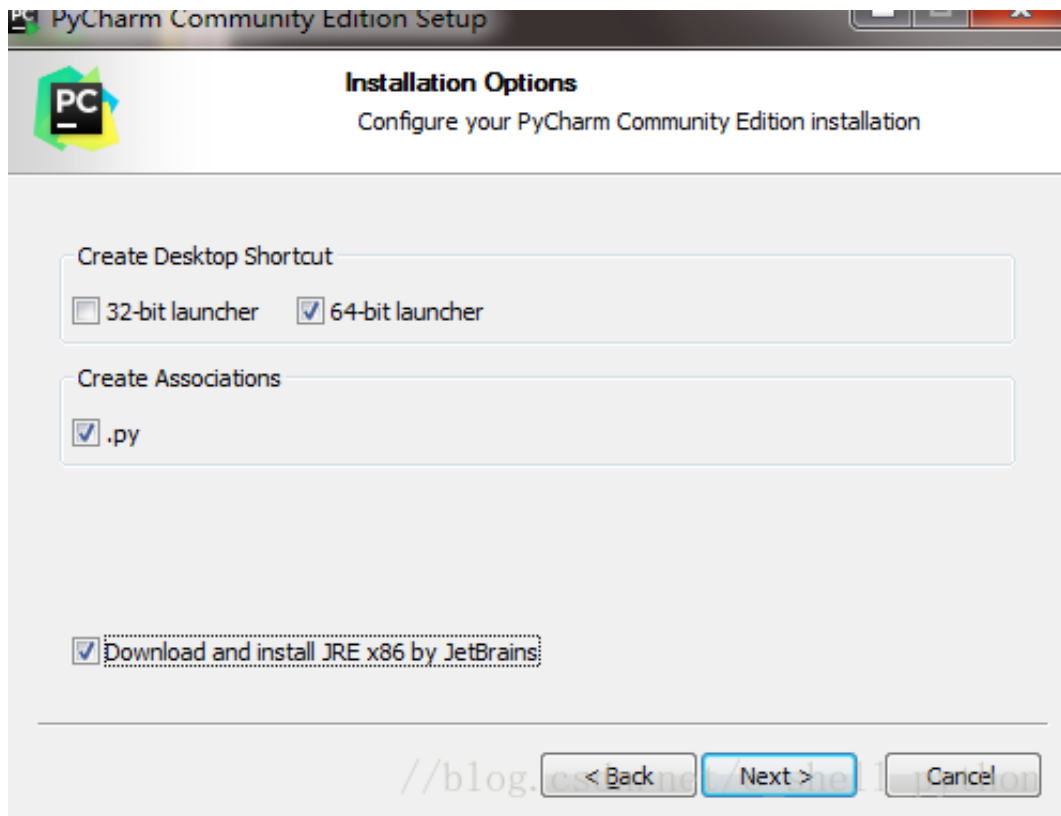


Get the Toolbox App to download PyCharm and its future updates with ease

- 1、当下载好以后，点击安装，记得修改安装路径，我这里放的是E盘，修改好以后，Next

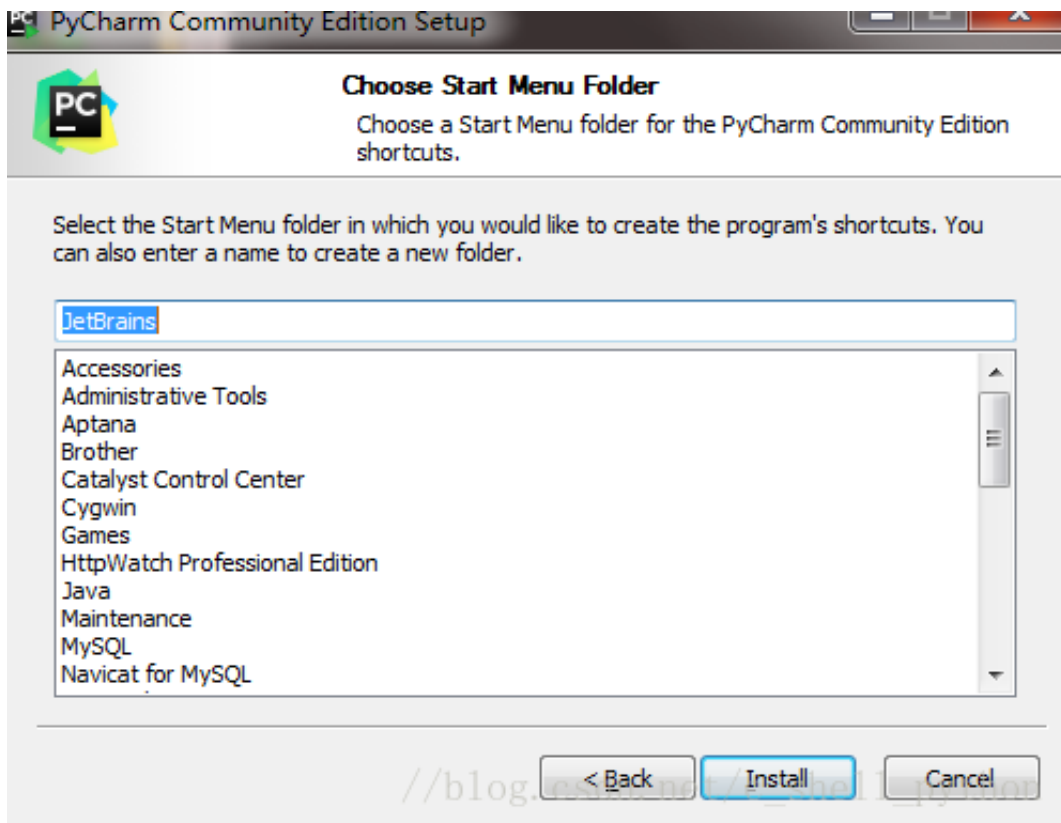


- 2、接下来是



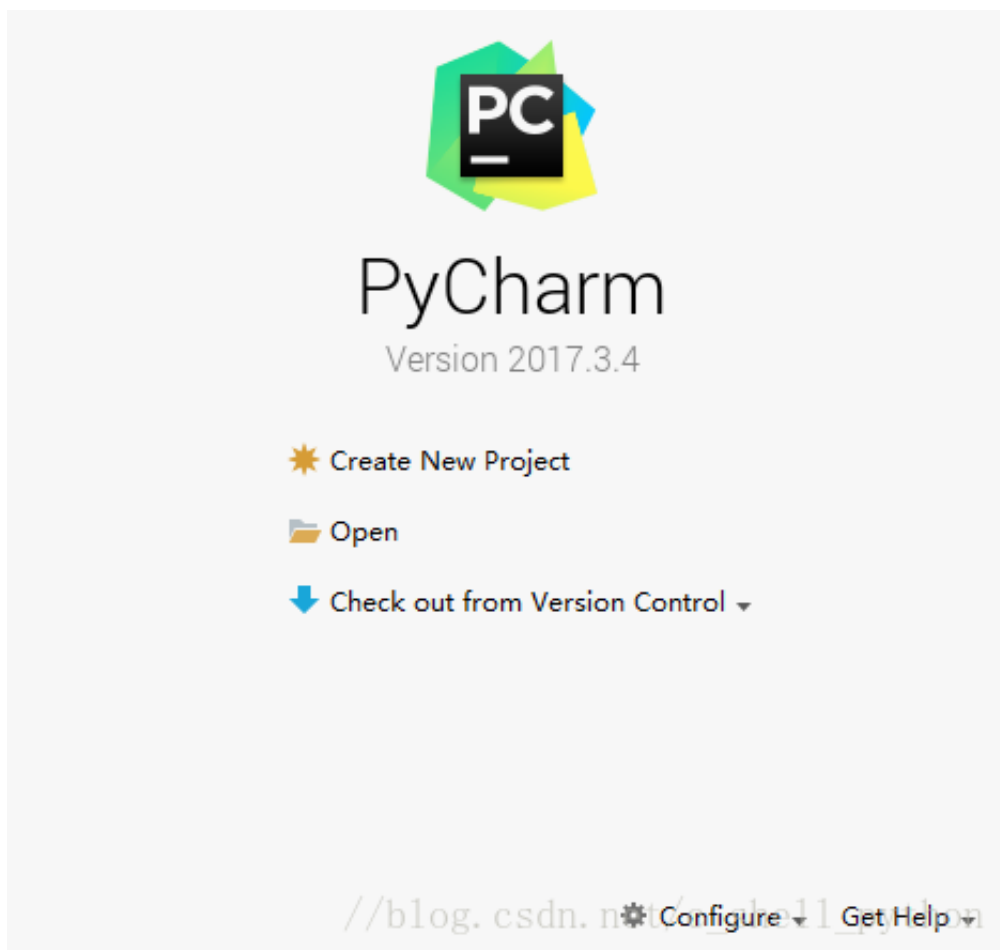
我们可以根据自己的电脑选择32位还是64位，目前应该基本都是64位系统吧

3、如下



点击Install,然后就是静静的等待安装了

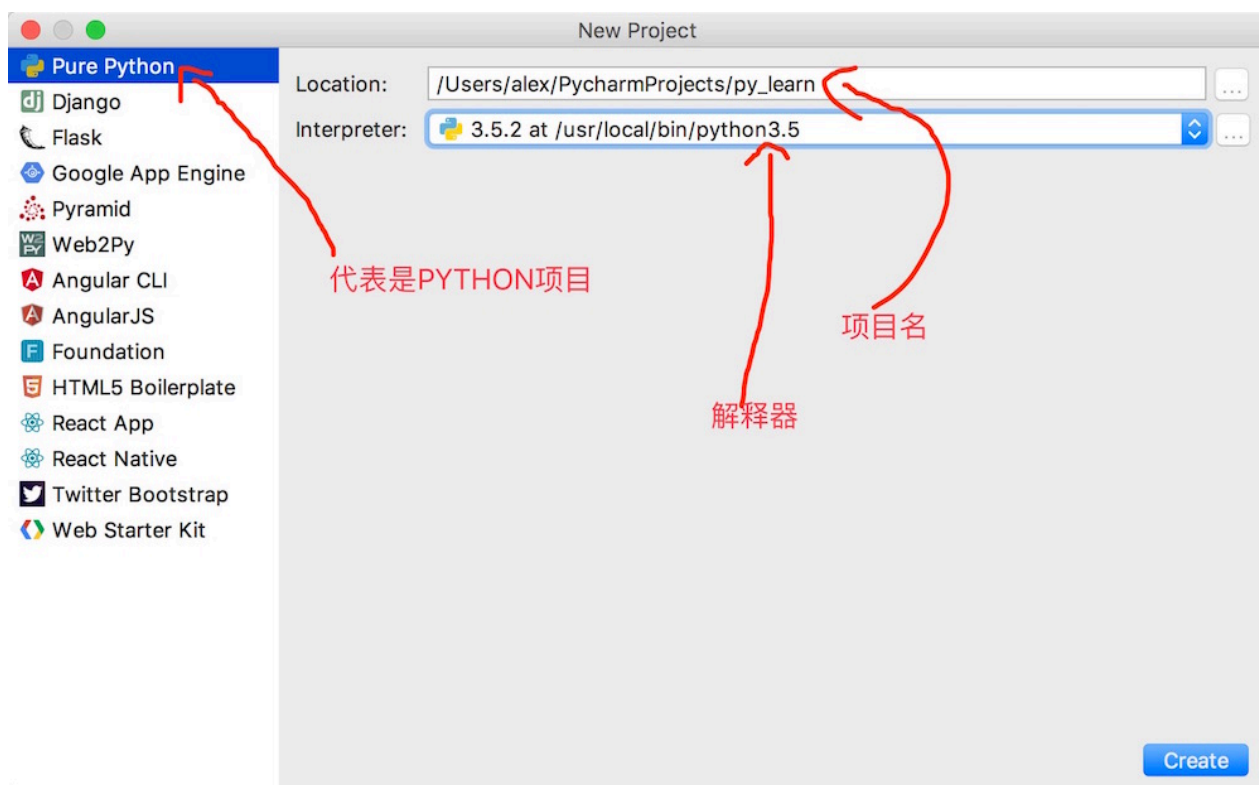
这时候Pycharm也装好了，我们进入该软件。



详细安装教程: <https://www.runoob.com/w3cnote/pycharm-windows-install.html>

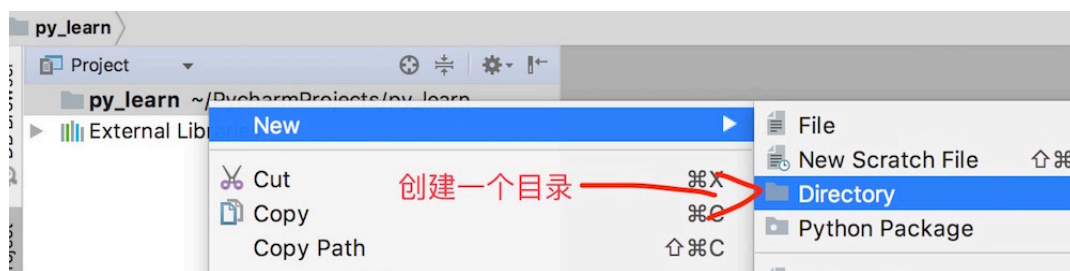
7.2 创建项目

注册完成后启动, 会让你先创建一个项目, 其实就是一个文件夹, 我们以后的代码都存在这里面

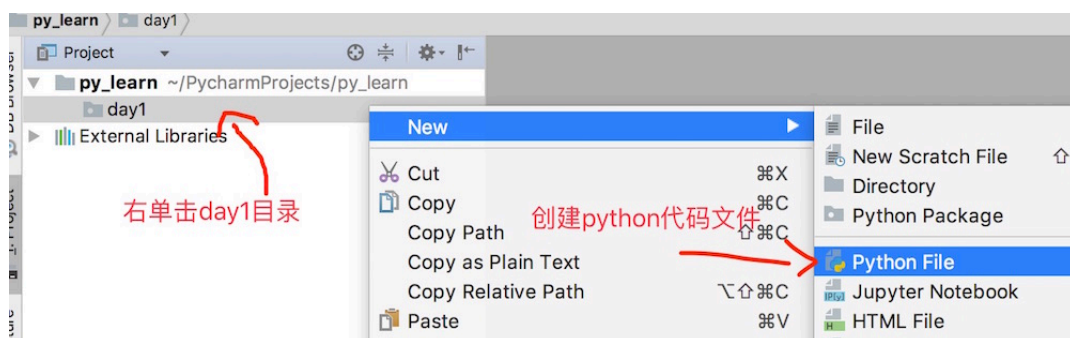


7.3 创建文件目录

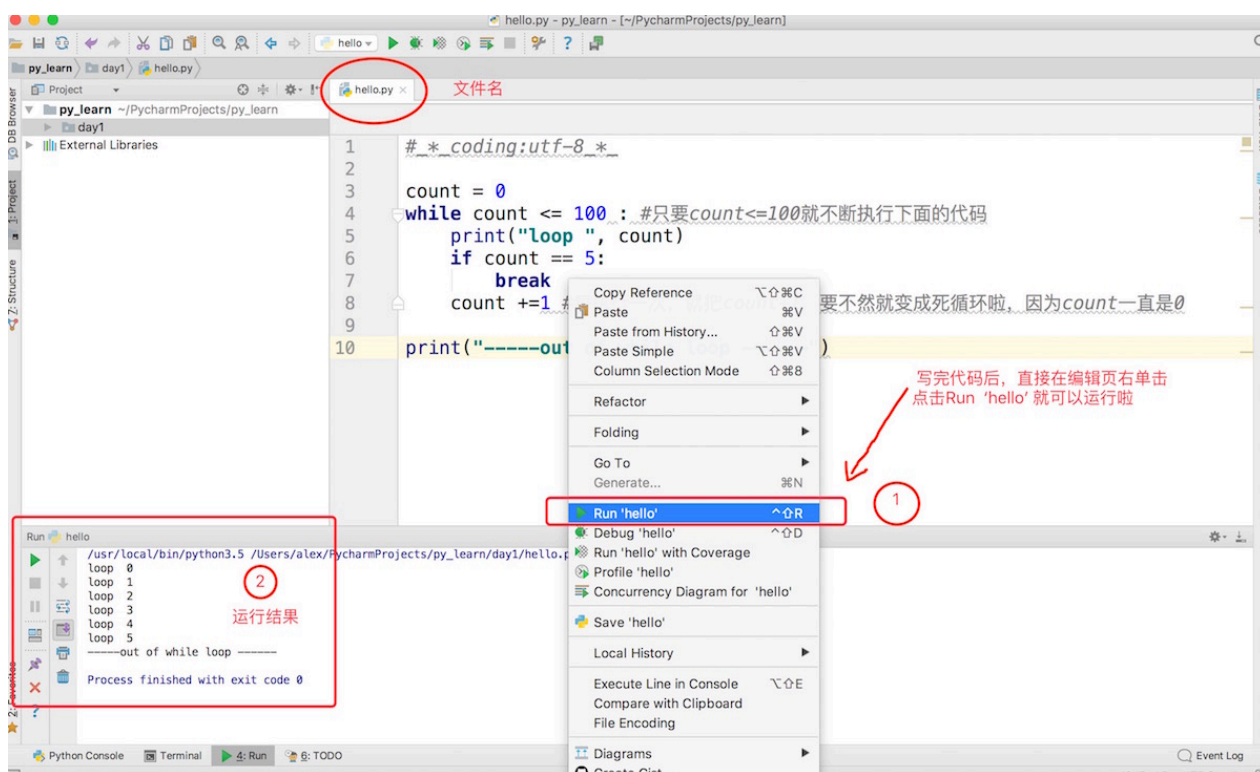
你以后写的项目可能有成百上千个代码文件，全放在一起可不好，所以一般把同样功能的代码放在一个目录，我们现在以天为单位，为每天的学习创建一个目录day1,day2,day3...这样



7.4 创建代码文件

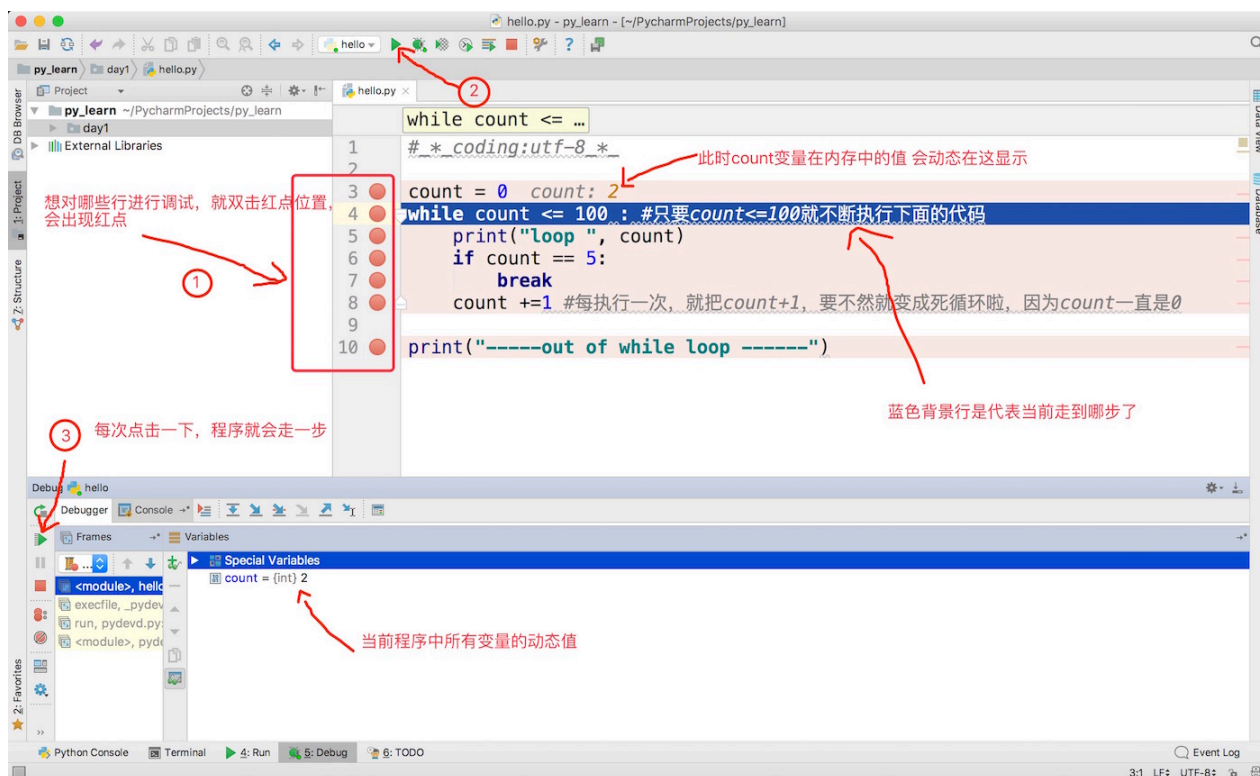


7.5 执行代码



7.6 代码调试

想不想看代码一步步的执行过程？比如想看每次循环某个变量有没有变化，总是print太low了，试试pycharm牛逼的debug功能吧



Pycharm还有很多更nb的功能，我们以后慢慢发掘

八、读取用户输入

如果你的程序要接收用户指令，可以用input语法：

```
name = input("What is your name?")
print("Hello " + name )
```

执行脚本就会发现，程序会等待你输入姓名后再往下继续走。

可以让用户输入多个信息，如下

```
name = input("What is your name?")
age = input("How old are you?")
hometown = input("Where is your hometown?")
print("Hello ", name , "your are ", age , "years old, you came from", hometown)
```

执行结果

```
What is your name?Alex Li
How old are you?22
Where is your hometown?ShanDong

Hello Alex Li your are 22 years old, you came from ShanDong
```

- 注意，input()方法接收的只是字符串，即使你输入的是数字，它也会按字符串处理

九、格式化打印

现有一练习题需求，问用户的姓名、年龄、工作、爱好，然后打印成以下格式

```
----- info of Alex Li -----
Name  : Alex Li
Age   : 22
job    : Teacher
Hobbie: girl
----- end -----
```

你怎么实现呢？你会发现，用字符拼接的方式很难实现这种格式的输出，所以一起来学一下新姿势

你只需要简单的在字符串里引用你外部的变量就可以啦。。。。。

语法

```
name = "Alex"
age = 22
print( f"hello my name is {name}, I am {age} old..")
```

- 注意字符串开头要加上 `f` ,才可以在里面用 `{}` 来引用外部变量

这样实现上面的需求就很简单啦

```
name = input("Name:")
age = input("Age:")
job = input("Job:")
hobbie = input("Hobbie:")

info = f'''
----- info of {name} -----
Name  : {name}
Age   : {age}
job    : {job}
Hobbie: {hobbie}
----- end -----
```

```
print(info)
```

十、流程控制之if...else...

假如把写程序比做走路，那我们到现在为止，一直走的都是直路，还没遇到过分叉口，想象现实中，你遇到了分叉口，然后你决定往哪拐必然是有所动机的。你要判断那条岔路是你真正要走的路，如果我们想让程序也能处理这样的判断怎么办？很简单，只需要在程序里预设一些条件判断语句，满足哪个条件，就走哪条岔路。这个过程就叫流程控制。

基本上在各个语言中，都是用语法if...else...来实现，可分为单分支、双分支、多分支

10.1 单分支

```
if 条件:
    满足条件后要执行的代码
```

```
salary = 8000    顿号：是固定语法，代码逻辑还没结束，下面缩进了的代码都受这个条件约束
if salary < 10000:
    print("你挣太少了，又长的丑，找对象有困难，建议来跟Alex学习Python+把妹技巧")
```

↑ 条件成立后，执行这条语句

4个空格，缩进必须有，原因晚点解释

```
if语句 ×
/Users/alex/Documents/work/PyProjects/自动化29期/venv/bin/python "/Users/alex/Documents/work/PyProjects/自动化29期/day1 F
你挣太少了，又长的丑，找对象有困难，建议来跟Alex学习Python+把妹技巧
Process finished with exit code 0
```

结果

10.2 双分支

```
if 条件:
    满足条件执行代码
else:
    if条件不成立就走这段代码
```

```
salary = 18000
if salary < 10000:
    print("你挣太少了，又长的丑，找对象有困难，建议来跟Alex学习Python+把妹技巧")
else:
    print("恭喜满足了在北京生存的基本收入线")
```

10.3 缩进

这里必须要插入这个缩进的知识点

你会发现，上面的if代码里，每个条件的下一行都缩进了4个空格，这是为什么呢？这就是Python的一大特色，强制缩进，目的是为了程序知道，每段代码依赖哪个条件，如果不通过缩进来区分，程序怎么会知道，当你的条件成立后，去执行哪些代码呢？

在其它的语言里，大多通过 `{ }` 来确定代码块，比如C,C++,Java,JavaScript都是这样，看一个JavaScript代码的例子

```
var age = 56
if ( age < 50 ){
    console.log("还能折腾")
    console.log('可以执行多行代码')
}else{
    console.log('太老了')
}
```

在有 `{ }` 来区分代码块的情况下，缩进的作用就只剩下让代码变的整洁了。

Python是门超级简洁的语言，发明者定是觉得用 `{ }` 太丑了，所以索性直接不用它，那怎么能区分代码块呢？答案就是强制缩进。

Python的缩进有以下几个原则：

- 顶级代码必须顶行写，即如果一行代码本身不依赖于任何条件，那它必须不能进行任何缩进
- 同一级别的代码，缩进必须一致
- 官方建议缩进用4个空格，当然你也可以用2个，如果你想被人笑话的话。

10.4 多分支

回到流程控制上来，if...else ...可以有多个分支条件

```
if 条件：
    满足条件执行代码
elif 条件： # else if ...
    满足条件执行代码,前提是上一层的if条件不满足
elif 条件：
    满足条件执行代码, 前提是上一层的elif条件不满足
elif 条件：
    满足条件执行代码,前提是上一层的elif条件不满足
else：
    上面所有的条件不满足就走这段
```

写个猜年龄的游戏吧

```
age_of_oldboy = 48
guess = int(input(">>:"))
if guess > age_of_oldboy :
    print("猜的太大了, 往小里试试...")
elif guess < age_of_oldboy :
    print("猜的太小了, 往大里试试...")
else:
    print("恭喜你, 猜对了...")
```

上面的例子, 根据你输入的值不同, 会最多得到3种不同的结果

再来实现一下下面这个需求吧, 写一段程序, 读取用户输入的工资, 根据工资多少打印相应的文字

你的工资多少决定了你的心态

月薪1000: 老板, 我是你爹。月薪2000: 老板, wqnm!gbxxxx 月薪5000: 老板脑子有坑, 背后说坏话。月薪1万: 老板说的有点问题, 但我不说话。月薪2万: 老板说啥就是啥吧, 给钱就行。月薪3万: 老板说什么都是对, 如果有人错了, 那一定是我。月薪5万: 996就像呼吸一样自然。月薪10万: 公司就是我家。

请先自己写, 写20分钟还写不出来再看我的

```
salary = int(input("Salary:"))

if salary > 100000:
    print("公司就是我的家")
elif salary > 50000:
    print("996就像呼吸一样自然")
elif salary > 30000:
    print("老板说什么都是对的, 如果有人错了, 那一定是我")
elif salary > 20000:
    print("老板说啥就是啥吧, 给钱就行")
elif salary > 10000:
    print("老板说的有点问题, 但是我不说话")
elif salary > 5000:
    print("老板脑子有坑, 背后说坏话")
elif salary > 2000:
    print("老板, wqnm!gbxxxx")
else:
    print("老板, 我是你爹...")
```

这里有个问题, 就是当我输入25000的时候, 它打印的结果是“老板说啥就是啥吧, 给钱就行”, 但是25000明明也大于第条件 `elif salary >= 10000:` 呀, 为什么不同时执行后面的呢呢? 这是因为代码是从上到下依次判断, 只要满足一个, 就不会再往下走啦, 这一点一定要清楚呀!

十一、今日练习题

11.2 写个匹配成绩的小程序，成绩有ABCDE 5个等级，与分数的对应关系如下

A	90-100
B	80-89
C	60-79
D	40-59
E	0-39

要求用户输入0-100的数字后，你能正确打印他的对应成绩等级，比如输入的是75，则打印C